RTOS PWM_NG

开发指南

版本号: 1.6

发布日期: 2025.8.12

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2023.3.19	XAA0311	初始版本
1.1	2024.2.23	XAA0311	添加 AI985 平台说明
1.2	2024.8.23	AWA2215	添加 MR536 平台说明
1.3	2024.9.9	AWA2215	添加 T536 平台说明
1.4	2025.5.26	AWA2231	添加 MR153 与 T153 平台说明添加 FAQ 内容
1.5	2025.7.25	AWA2231	添加 F135 平台说明
1.6	2025.8.12	AWA2231	完善文档格式

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1
1.4 相关术语介绍 1

2 模块介绍 **3**
2.1 模块功能介绍 3
2.2 模块配置介绍 3
2.2.1 设备配置说明 3
2.2.2 kernel menuconfig 配置说明 6
2.3 模块源码结构 6

3 模块接口说明 **7**
3.1 接口列表 7
3.2 接口使用说明 7
3.2.1 不同平台 pwm 资源介绍 7
3.2.2 PWM 初始化接口 8
3.2.3 PWM 控制接口 8
3.2.4 单通道控制接口 8
3.2.5 双通道控制接口 8

4 模块使用范例 **10**

5 测试 **13**
5.1 基本功能测试 13
5.2 PWM 对插入死区测试 14
5.3 捕获功能测试 16
5.4 捕获计数功能测试 16

6 FAQ **18**
6.1 当 IO 口未输出波形或者输出的波形与设置不匹配 18

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 5-1　测试 1 13
图 5-2　pwm 波形图 1 14
图 5-3　死区设置 15
图 5-4　pwm4 死区波形图 16
图 5-5　测试 2 16
图 5-6　pwm 捕获计数测试 17
图 6-1　dump PD2 的寄存器 18
图 6-2　PWM PCR 寄存器 bit0-bit7 说明 18
图 6-3　PWM PCR 寄存器 bit8-bit31 说明 19
图 6-4　PWM PPR 寄存器说明 20
图 6-5　PWM PCR 和 PPR 寄存器 20

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

介绍 RTOS 中 PWM 驱动的接口及使用方法，为 PWM 使用者提供参考。

1.2 目标读者

PWM 驱动的开发/维护人员。

1.3 适用范围

表 1：适用产品列表

产品名称	内核版本	| 驱动文件
MR527	FreeRTOS	sunxi_pwm_ng.c
AI985	FreeRTOS	sunxi_pwm_ng.c
MR536	FreeRTOS	sunxi_pwm_ng.c
T536	FreeRTOS	sunxi_pwm_ng.c
MR153	FreeRTOS	sunxi_pwm_ng.c
T153	FreeRTOS	sunxi_pwm_ng.c
F135	FreeRTOS	sunxi_pwm_ng.c

1.4 相关术语介绍

版权所有 © 珠海全志科技股份有限公司。保留一切权利

术语	解释说明
频率	PWM 的频率决定了所模拟电平的平滑度（逼真度），人耳感知的频率范围为 20Hz-16 kHz，注意 PWM 的频率不要落在这个区间。
占空比	决定了一个周期内 PWM 信号高低的比例，进而决定了一个周期内的平均电压，也就是所模拟的电平的电压。
极性	决定了是高占空比的信号输出电平高，还是低占空比信号输出电平高。假设一个信号的占空比为 100%，如果为正常极性，则输出电平最大，如果为翻转的极性，则输出电平为 0。
开关	控制 PWM 信号是否输出。
PWM 对	电机等硬件需要两路脉冲信号来控制其正常运转，一般两路极性相关，频率，占空比参数相同的 PWM 构成一个 PWM 对。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 模块介绍

2.1 模块功能介绍

脉冲宽度调制（PWM）是一种对模拟信号电平进行数字编码的方法。通过高分辨率计数器的使用，方波的占空比被调制用来对一个具体模拟信号的电平进行编码。PWM 具有以下规格特性：

- 支持脉冲（脉冲个数可配）、周期和互补对输出；
- 支持捕捉输入；
- 带可编程死区发生器，死区时间可控；
- 0-24 MHz/100 MHz 输出频率范围、0%-100% 占空比可调、最小分辨率 1/65536（即 1/2^16）；
- 支持 PWM 输出和捕捉输入产生中断；
- 支持 PWM 组模式，同组内各个通道起始相位可配置。

2.2 模块配置介绍

2.2.1 设备配置说明

PWM 模块寄存器的基本配置位于 sdk/lichee/rtos-hal/hal/source/pwm_ng/platform/pwm_sun55iw3.h。

```
/* pwm 各个资源的基地址 */
#define PWM0_CPUX_BASE    0x02000c00UL
#define PWM1_CPUX_BASE    0x02051000UL
#define PWM_CPUS_BASE     0x07020c00UL
#define PWM_DSP_BASE      0x07103000UL

#define PWM_CHAN_NUM      16    /* 一套资源支持的最多通道数目 */
#define PWM_PORT_NUM      4     /* pwm支持资源的最大套数 */
/* CPUX PWM0 config */
#define SUNXI_CPUX_PWM0_CLK_TYPE    HAL_SUNXI_CCU
#define SUNXI_CPUX_PWM0_CLK_BUS_ID  0
#define SUNXI_CPUX_PWM0_CLK_ID      CLK_PWM
#define SUNXI_CPUX_PWM0_RESET_TYPE  HAL_SUNXI_RESET
#define SUNXI_CPUX_PWM0_RESET_ID    RST_BUS_PWM
#define PWM0_CPUX_BASE              0x02000c00UL
#define SUNXI_CPUX_IRQ_PWM0         MAKE_IRQn(92, 4)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
/* CPUX PWM1 config */
#define SUNXI_CPUX_PWM1_CLK_TYPE    HAL_SUNXI_CCU
#define SUNXI_CPUX_PWM1_CLK_ID      CLK_PWM1
#define SUNXI_CPUX_PWM1_CLK_BUS_ID  0
#define SUNXI_CPUX_PWM1_RESET_TYPE  HAL_SUNXI_RESET
#define SUNXI_CPUX_PWM1_RESET_ID    RST_BUS_PWM1
#define PWM1_CPUX_BASE              0x02051000UL
#define SUNXI_CPUX_IRQ_PWM1         MAKE_IRQn(107, 7)

/* CPUS PWM config */
#define SUNXI_CPUS_PWM_CLK_TYPE     HAL_SUNXI_R_CCU
#define SUNXI_CPUS_PWM_CLK_BUS_ID   CLK_BUS_R_PWM
#define SUNXI_CPUS_PWM_CLK_ID       CLK_R_PWM
#define SUNXI_CPUS_PWM_RESET_TYPE   HAL_SUNXI_R_RESET
#define SUNXI_CPUS_PWM_RESET_ID     RST_R_PWM
#define SUNXI_CPUS_IRQ_PWM          MAKE_IRQn(72, 0)
#define PWM_CPUS_BASE               0x07020c00UL

/* DSP PWM config */
#define SUNXI_DSP_PWM_CLK_TYPE      HAL_SUNXI_DSP
#define SUNXI_DSP_PWM_CLK_BUS_ID    CLK_BUS_DSP_PWM
#define SUNXI_DSP_PWM_CLK_ID        CLK_DSP_PWM
#define SUNXI_DSP_PWM_RESET_TYPE    HAL_SUNXI_DSP_RESET
#define SUNXI_DSP_PWM_RESET_ID      RST_BUS_DSP_PWM
#define SUNXI_DSP_IRQ_PWM           MAKE_IRQn(17, 0)
#define PWM_DSP_BASE                0x07103000UL

/* 不同通道的资源整合 */
#define CPUX_PWM0_PARAMS \
{    .port = 0, \
     .reg_base = PWM0_CPUX_BASE, .irq_num = SUNXI_CPUX_IRQ_PWM0, .gpio_num = 16, \
     .clk_type = SUNXI_CPUX_PWM0_CLK_TYPE, .bus_clk_id = SUNXI_CPUX_PWM0_CLK_BUS_ID, \
     .clk_id = SUNXI_CPUX_PWM0_CLK_ID, .reset_type = SUNXI_CPUX_PWM0_RESET_TYPE, \
     .reset_id = SUNXI_CPUX_PWM0_RESET_ID, \
}

#define CPUX_PWM1_PARAMS \
{    .port = 1, \
     .reg_base = PWM1_CPUX_BASE, .irq_num = SUNXI_CPUX_IRQ_PWM1, .gpio_num = 4, \
     .clk_type = SUNXI_CPUX_PWM1_CLK_TYPE, .bus_clk_id = SUNXI_CPUX_PWM1_CLK_BUS_ID, \
     .clk_id = SUNXI_CPUX_PWM1_CLK_ID, .reset_type = SUNXI_CPUX_PWM1_RESET_TYPE, \
     .reset_id = SUNXI_CPUX_PWM1_RESET_ID, \
}

#define CPUS_PWM_PARAMS \
{    .port = 2, \
     .reg_base = PWM_CPUS_BASE, .irq_num = SUNXI_CPUS_IRQ_PWM, .gpio_num = 2, \
     .clk_type = SUNXI_CPUS_PWM_CLK_TYPE, .bus_clk_id = SUNXI_CPUS_PWM_CLK_BUS_ID, \
     .clk_id = SUNXI_CPUS_PWM_CLK_ID, .reset_type = SUNXI_CPUS_PWM_RESET_TYPE, \
     .reset_id = SUNXI_CPUS_PWM_RESET_ID, \
}

#define DSP_PWM_PARAMS \
{    .port = 3, \
     .reg_base = PWM_DSP_BASE, .irq_num = SUNXI_DSP_IRQ_PWM, .gpio_num = 8, \
     .clk_type = SUNXI_DSP_PWM_CLK_TYPE, .bus_clk_id = SUNXI_DSP_PWM_CLK_BUS_ID, \
     .clk_id = SUNXI_DSP_PWM_CLK_ID, .reset_type = SUNXI_DSP_PWM_RESET_TYPE, \
     .reset_id = SUNXI_DSP_PWM_RESET_ID, \
}
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
/*****************************************************************************
 * define  gpio
 *****************************************************************************/
typedef struct pwm_gpio_t
{
    gpio_pin_t pwm_pin;
    int  pwm_function;
    bool bind_mode;
    u32 bind_channel;
    u32 dead_time;
}  pwm_gpio_t;

/*  各个资源及对应的引脚配置 */
static  pwm_gpio_t pwm_gpio[PWM_PORT_NUM][PWM_CHAN_NUM] =
{
    {
        /*  CPUX0 PWM0 */
        {
            /*  pwm0 */
            . pwm_pin = GPIOD(0),
            . pwm_function = 5,
        },
    },

    {
        /*  CPUX1 PWM1 */
        {
            /*  pwm0 */
            . pwm_pin = GPIOD(16),
            . pwm_function = 5,
        },
    },

    {
        /*  CPUS PWM */
        {
            /*  pwm0 */
            . pwm_pin = GPIOL(10),
            . pwm_function = 2,
        },
    },

    {
        /*  DSP PWM */
        {
            /*  pwm0 */
            . pwm_pin = GPIOL(2),
            . pwm_function = 4,
        },
    }
};
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.2 kernel menuconfig 配置说明

```
Drivers Options  --->
 soc related device drivers  --->
   SUNXI PWM Devices
     [*]enable pwm-ng driver
     [*]enable pwm-ng hal APIs test command
```

2.3 模块源码结构

PWM 模块源码结构如下所示：

```
hal/source/pwm_ng/ ---- 驱动源码
 ├──── hal_pwm_ng.c
 ├──── Kconfig
 ├──── Makefile
 ├──── platform
 │     └──── pwm_sun8iw22.h
 │     └──── pwm_sun55iw3.h
 │     └──── pwm_sun55iw6.h
 └──── platform_pwm_ng.h

include/hal/ ---- 驱动APIs声明头文件
 └──── sunxi_hal_pwm_ng.h

hal/test/pwm_ng/ ---- 驱动APIs测试代码
 ├──── Makefile
 └──── test_pwm_ng.c
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 模块接口说明

需要包含头文件:

```
#include <sunxi_hal_pwm_ng.h>
```

3.1 接口列表

PWM 提供的接口列表如下：

```
pwm_status_t hal_pwm_init(hal_pwm_port_t port)
pwm_status_t hal_pwm_control(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
pwm_status_t hal_pwm_control_single(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
pwm_status_t hal_pwm_control_dual(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
```

3.2 接口使用说明

3.2.1 不同平台 pwm 资源介绍

产品名称	pwm 所属处理单元	port 编号	pwm 通道数量
MR527/AI985	CPUX	0	16
MR527/AI985	CPUX	1	16
MR527/AI985	CPUS	2	2
MR527/AI985	DSP	3	8
MR536/T536	CPUX	0	10
MR536/T536	CPUX	1	10
MR536/T536	CPUX	2	10
MR536/T536	CPUS	3	4
MR153/T153	CPUX	0	8
MR153/T153	CPUX	1	8

版权所有 © 珠海全志科技股份有限公司。保留一切权利

产品名称	pwm 所属处理单元	port 编号	pwm 通道数量
MR153/T153	CPUX	2	10
F135	CPUX	0	8

3.2.2 PWM 初始化接口

- 函数原型：pwm_status_t hal_pwm_init(hal_pwm_port_t port)
- 功能描述：程序运行的入口函数。
- 参数说明：port, 指使用哪套资源（CPUX、CPUS、DSP）。
- 返回值：=0, 运行成功；<0 , 失败。

3.2.3 PWM 控制接口

- 函数原型：pwm_status_t hal_pwm_control(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
- 功能描述：完成对 pwm 通道的控制。
- 参数说明：port, 指使用哪套资源（CPUX、CPUS、DSP），channel, 指向所属的 pwm 通道；config_pwm，pwm_config 类型的指针, 包含周期、极性、占空比。
- 返回值：=0, 运行成功；<0 , 失败。

3.2.4 单通道控制接口

- 函数原型：pwm_status_t hal_pwm_control_single(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
- 功能描述：PWM 多通道控制函数。
- 参数说明：port, 指使用哪套资源（CPUX、CPUS、DSP），channel, 指向所属的 pwm 通道；config_pwm，pwm_config 类型的指针, 包含周期、极性、占空比。
- 返回值：=0, 运行成功；<0 , 失败。

3.2.5 双通道控制接口

- 函数原型：pwm_status_t hal_pwm_control_dual(hal_pwm_port_t port, int channel, struct pwm_config *config_pwm)
- 功能描述：PWM 多通道控制函数。
- 参数说明：port, 指使用哪套资源（CPUX、CPUS、DSP），channel, 指向所属的 pwm 通道；config_pwm，pwm_config 类型的指针, 包含周期、极性、占空比。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

- 返回值：=0, 运行成功；<0 , 失败。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 模块使用范例

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <unistd.h>

#include <hal_log.h>
#include <hal_cmd.h>
#include <sunxi_hal_pwm_ng.h>

#if defined(CONFIG_ARCH_DSP)
#include <delay.h>
#define sleep(sec) msleep(sec * 1000)
#endif

#ifndef printfFromISR
#define printfFromISR printf
#endif

static int cmd_test_pwm(int argc, char **argv)
{
    struct pwm_config *config;
    uint8_t port;
    uint8_t channel;
    int period, duty;

    if (argc < 5) {
        hal_log_info("Usage: port | pwm channel | duty | period\n");
        return -1;
    }

    hal_log_info("Run pwm hal layer test case\n");

    // 解析命令行参数：PWM资源编号
    port = strtol(argv[1], NULL, 0);
    hal_log_info("port = %d", port);
    // 解析命令行参数：PWM通道号
    channel = strtol(argv[2], NULL, 0);
    hal_log_info("channel = %d", channel);
    // 解析命令行参数：占空比时间（纳秒）
    duty = strtoul(argv[3], NULL, 0);
    // 解析命令行参数：周期时间（纳秒）
    period = strtoul(argv[4], NULL, 0);

    // 为PWM配置结构体分配内存
    config = (struct pwm_config *)malloc(sizeof(struct pwm_config));

    // 设置PWM配置参数
    config->duty_ns  = duty;      // 设置占空比时间（纳秒）
    config->period_ns = period;   // 设置周期时间（纳秒）
    config->polarity  = PWM_POLARITY_NORMAL; // 设置极性为正常极性
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    hal_log_info("duty_ns = %d \n", config->duty_ns);
    hal_log_info("period_ns = %d \n", config->period_ns);
    hal_log_info("polarity = %d \n", config->polarity);

    // 初始化指定PWM资源
    hal_pwm_init(port);

    // 控制指定PWM通道输出配置的波形
    hal_pwm_control(port, channel, config);

    hal_log_info("control pwm test finish\n");

    return 0;
}

FINSH_FUNCTION_EXPORT_CMD(cmd_test_pwm, hal_pwm, pwm hal APIs tests)

static int cmd_release_pwm_channel(int argc, char **argv)
{
    struct pwm_config *config;
    uint8_t port;
    uint8_t channel;
    uint8_t ns;
    ulong period, duty;

    if (argc < 3) {
        hal_log_info("Usage: pwm port\n");
        return -1;
    }

    hal_log_info("Run close pwm channel test case\n");

    // 解析命令行参数：PWM资源编号
    port = strtol(argv[1], NULL, 0);
    // 解析命令行参数：PWM通道号
    channel = strtol(argv[2], NULL, 0);
    hal_log_info("channel = %d", channel);

    // 释放指定PWM资源中的指定通道
    hal_pwm_release(port, channel);

    hal_log_info("release pwm channel finish\n");

    return 0;
}

FINSH_FUNCTION_EXPORT_CMD(cmd_release_pwm_channel, hal_pwm_close, release pwm channel hal APIs tests)

static void pwm_cap_callback(void* param)
{
    // PWM捕获回调函数，用于处理捕获到的PWM信号信息
    hal_pwm_cap_info *info = (hal_pwm_cap_info *)param;

    // 打印捕获到的PWM信号参数：通道号、计数值、周期、占空比
    printfFromISR("pwm%d capture callback, cnt is %d, period is %d, duty is %d\n", info->channel, info->cnt, info->period
        , info->duty);
}

hal_pwm_status_t pwm_capture_init(uint8_t port, uint32_t channel)
{
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
    // 初始化PWM捕获功能，启用指定端口和通道的捕获
    // port: PWM资源编号，channel: PWM通道号
    // pwm_cap_callback: 捕获回调函数，当捕获到PWM信号时调用
    hal_pwm_cap_enable(port, channel, pwm_cap_callback);

    return HAL_PWM_STATUS_OK;
}

hal_pwm_status_t pwm_capture_deinit(uint8_t port, uint32_t channel)
{
    // 停用PWM捕获功能，释放指定端口和通道的捕获资源
    // port: PWM资源编号，channel: PWM通道号
    hal_pwm_cap_disable(port, channel);

    return HAL_PWM_STATUS_OK;
}

void cmd_pwm_capture_help(void)
{
    // 打印PWM捕获测试的帮助信息
    printf("pwm capture test, stop capture after 10s\n");
    printf("usage: pwm_capture_test<channel>\n");
    printf("\t<channel>: 0 ~ 15\n");
    printf("eg: pwm_capture_test 0, pwm0 capture function\n");
}

int cmd_pwm_capture(int argc, char *argv[])
{
    int count = 0;
    uint8_t port;
    uint8_t channel;

    // 解析命令行参数：PWM资源编号
    port = strtol(argv[1], NULL, 0);
    // 解析命令行参数：PWM通道号
    channel = strtol(argv[2], NULL, 0);
    if (argc != 3 || channel > 15) {
        cmd_pwm_capture_help();
        return -1;
    }

    /* capture setting */
    // 初始化PWM捕获功能
    pwm_capture_init(port, channel);

    // 等待10秒，期间持续捕获PWM信号
    for(count = 0; count < 10; count++) {
        sleep(1); // 每次休眠1秒
    }

    /* disable */
    // 停用PWM捕获功能，释放资源
    pwm_capture_deinit(port, channel);

    printf("[%s]: pwm stop capture ssd\n", __func__);

    return 0;
}

FINSH_FUNCTION_EXPORT_CMD(cmd_pwm_capture, pwm_capture_test, pwm capture test)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 测试

5.1 基本功能测试

注意：运行hal_pwm 0 0 500 1000，其中，0 0 500 1000对应[PWM资源][PWM通道] [脉冲] [周期]。

执行hal_pwm 0 0 500 1000的关键 log 和波形图如下：

```
cpu0>hal_pwm 0 0 500 1000
Run pwm hal layer test case

port = 0
channel = 0
duty_ns = 500

period_ns = 1000

polarity = 1

[pwm0] not support in sys_config

control pwm test finish
```

图 5-1: 测试 1

版权所有 © 珠海全志科技股份有限公司。保留一切权利

图 5-2: pwm 波形图 1

5.2 PWM 对插入死区测试

死区的配置如下所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
352             {
353                     /* pwm2 */
354                     .pwm_pin = GPIOL(4),
355                     .pwm_function = 4,
356                     .bind_mode = 1,
357                     .bind_channel = 3,
358                     .dead_time = 20,
359             },
360             {
361                     /* pwm3 */
362                     .pwm_pin = GPIOL(5),
363                     .pwm_function = 4,
364             },
365             {
366                     /* pwm4 */
367                     .pwm_pin = GPIOL(6),
368                     .pwm_function = 3,
369                     .bind_mode = 1,
370                     .bind_channel = 5,
371                     .dead_time = 20,
372             },
373             {
374                     /* pwm5 */
375                     .pwm_pin = GPIOL(7),
376                     .pwm_function = 3,
377             },
378             {
379                     /* pwm6 */
```

图 5-3: 死区设置

注意：运行hal_pwm 4 2 8000 20000，其中，4 2 8000 20000对应[PWM资源][PWM通道] [脉冲(ns)] [周期(ns)]。**参数计算逻辑说明：**- 脉冲宽度 = 8000 ns = 8 µs- 周期时间 = 20000 ns = 20 µs- 占空比 = (脉冲宽度/周期时间) × 100% = (8000/20000) × 100% = 40%- 频率 = 1/周期时间 = 1/20 µs = 50 kHz

执行hal_pwm 4 2 8000 20000波形图如下：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

图 5-4: pwm4 死区波形图

5.3 捕获功能测试

注意：运行pwm_capture_test 0 0，其中，0 0对应[PWM资源][PWM通道]。

执行hal_pwm 0 0的关键 log 如下：

```
cpu0>pwm_capture_test 0 0
[pwm0] not support in sys_config

[cmd_pwm_capture]: pwm stop capture ssd

cpu0>
```

图 5-5: 测试 2

5.4 捕获计数功能测试

表 2：功能适用产品列表

产品名称	内核版本	驱动文件
MR536		

广州星翼电子科技有限公司zdyzlinux

| FreeRTOS | sunxi_pwm_ng.c | +————————+————————–+————————–+ | T536 | FreeRTOS | sunxi_pwm_ng.c | +————————+————————–+————————–+ | MR153 | FreeRTOS | sunxi_pwm_ng.c | +————————+————————–+————————–+ | F135 | FreeRTOS | sunxi_pwm_ng.c | +————————+————————–+————————–+ 以 pwm0-0 作为输出，pwm1-0 作为捕获为例，将 pwm0-0 和 pwm1-0 引脚短接。

1、pwm0-0 输出 1KHZ，执行：

```
hal_pwm 0 0 500000 1000000 0 0 0
```

2、pwm1-0 捕获 1 秒脉冲个数，执行：

```
pwm_capture_count_test 1 0 1
```

> usgae: pwm_capture_count_test

关键 log 如下：

```
cpu0>hal_pwm 0 0 500000 1000000 0 0 0
Run pwm hal layer test case in cycle mode!

duty_ns = 500000

period_ns = 1000000

polarity = 0

pulse_num = 0

output_mode = 0

control pwm test finish


cpu0>pwm_capture_count_test 1 0 1
[cmd_pwm_capture_count]: pulse_num is: 1000
```

图 5-6: pwm 捕获计数测试

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 FAQ

6.1 当 IO 口未输出波形或者输出的波形与设置不匹配

可做如下排查：

① 排查 PWM 的 IO 状态

通过在 RV 端输入如下命令，将输出 io 的复用功能 dump 出来，对比是否一致，检查引脚的配置，例如通过查询《**MR527_User_Manual**》得知 PD2 的功能引脚寄存器为 0x02000090，打印的寄存器值如下：

```
cpu0>p 0x02000090 4
start_addr=0x02000090 end_addr=0x02000094
reg_addr[0x02000090]=0xfffff5f5
reg_addr[0x02000094]=0xffffffff
```

图 6-1: dump PD2 的寄存器

关注第 8 到 11bit 为 PD2 的功能引脚参数，并可参照 2.2.2.1 的 PD2 功能引脚寄存器表，第 8 到 11bit 的值为 5 对应 PWM 功能。

② 排查 PWM 的寄存器状态

dump PWM 的功能设置寄存器 PCR 和周期占空比寄存器 PPR，确认设置的参数是否写入到寄存器中，寄存器的相关介绍如下：

Offset: 0x0100+N*0x0020			Register Name: PCR
Bit	Read/Write	Default/Hex	Description
7:0	R/W	0x0	PWM_PRESCAL_K PWM pre-scale K, actual pre-scale is (K+1) K = 0, actual pre-scale: 1 K = 1, actual pre-scale: 2 K = 2, actual pre-scale: 3 K = 3, actual pre-scale: 4 ..... K = 255, actual pre-scale: 256

图 6-2: PWM PCR 寄存器 bit0-bit7 说明

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Offset: 0x0100+N*0x0020			Register Name: PCR
Bit	Read/Write	Default/Hex	Description
31:16	R/W	0x0	PWM_PUL_NUM In pulse mode, PWM output pulse for PWM_CYCLE_NUM+1 times and then stop.
15:12	/	/	/
11	R	0x0	PWM_PERIOD_RDY PWM period register ready 0: PWM period register is ready to write 1: PWM period register is busy.
10	R/WAC	0x0	PWM_PUL_START PWM pulse output start 0: no effect 1: output pulse for PWM_CYCLE_NUM+1 After finishing configuration for outputting pulse, set this bit once and then PWM would output waveform. After the waveform is finished, the bit will be cleared automatically.
9	R/W	0x0	PWM_MODE PWM output mode select 0: cycle mode 1: pulse mode
8	R/W	0x0	PWM_ACT_STA PWM active state 0: Low Level 1: High Level

图 6-3: PWM PCR 寄存器 bit8-bit31 说明

关注 PCR 寄存器以下 bit 位：

- 第 0~7bit 位为 cycle 的缩放比例，用于计算 PPR 寄存器的实际值。
- 第 8bit 位为极性配置，0 表示低电平为 active 状态，1 表示高电平为 active 状态。
- 第 9bit 位为模式配置，0 表示周期模式输出，1 表示脉冲模式输出。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Offset: 0x0104+N*0x0020			Register Name: PPR
Bit	Read/Write	Default/Hex	Description
31:16	R/W	0x0	PWM_ENTIRE_CYCLE Number of the entire cycles in the PWM clock 0 = 1 cycle 1 = 2 cycles …… N = N+1 cycles If the register need to be modified dynamically, the PCLK should be faster than the PWM CLK .
15:0	R/W	0x0	PWM_ACT_CYCLE Number of the active cycles in the PWM clock 0 = 0 cycle 1 = 1 cycles …… N = N cycles

图 6-4: PWM PPR 寄存器说明

关注 PPR 寄存器以下 bit 位：

- 第 0 到 15bit 位为 active 状态所占的周期，以 **cycles 的数量**表示对应周期时间。
- 第 16 到 31bit 位为整个周期，以 **cycles 的数量**表示对应周期时间。

以上面 PWM0_2 输出为例，dump 对应的 0x2000d40 PCR 寄存器和 0x2000d44 PPR 寄存器，可使用命令

```
p 0x02000d40 4
```

dump 寄存器指令如下：

```
cpu0>p 0x2000d40 4
start_addr=0x02000d40 end_addr=0x02000d44
reg_addr[0x02000d40]=0x00000103
reg_addr[0x02000d44]=0xea5f7530
```

图 6-5: PWM PCR 和 PPR 寄存器

其中 PPR 寄存器中 cycle 的数量和设置周期时间的计算公式如下：

cycle number = （周期时间 x 时钟频率）/ (PRESCAL x 1000000000)

周期时间为 5ms，时钟频率为 24M，PRESCAL 为寄存器 PCR 第 0 ~ 7bit 值，寄存器值 3 对应值为 4；

版权所有 © 珠海全志科技股份有限公司。保留一切权利

可计算 active 的 cycle 的数量对应 =（5000000 * 24000000）/ (4 * 1000000000) = 30000；对比 active 的寄存器 cycle 0x7530，配置与计算值一致，整个周期的 cycle 数量计算一样。

说明

公式的相关计算过程介绍如下：

由于设置的周期时间是以 ns 为单位，时钟频率是 1/s，在计算对应的 cycle 数量时，需要进行单位换算除于 10 的 9 次方，由于 cycle 数量值在寄存器里是有上限的，因此引入 cycle 的倍频 PRESCAL ，增大 cycle 数量的上限。

其中周期时间为自定义的时间，单位是 ns，时钟频率为 PWM 控制器所使用的频率，默认为 24M 时钟，当设置的周期小于 41.6ns（1/24M）时，改用 100M 时钟（驱动会根据周期时间自动切换选择时钟，无需手动设置）；时钟的选择可 dump PCCR 寄存器（0x02000c24）对比，具体介绍详见《MR527_User_Manual》，这里不再赘述。

从 PPR 寄存器可看出，数据为 16 位（最大值为 65535）；cycle 的数量对应周期时间，假如 cycle 与频率时间无倍增关系，则 65535 对应最大时间为 65535*（1/24M）与等于 2.72ms，明显不能满足我们之前 10ms 为周期的输出，因此引入 cycle 的倍增机制，增大每个 cycle 对应的时间，倍增系数为 PCR 0 ~ 7bit 位值。以之前 10ms 周期为例，2.72ms 需倍增 4 倍可达 10ms，因此 PCR 的 0 ~ 7bit 理论计算为 3（寄存器值为 3 实际增倍为 3+1=4）；从寄存器可得出 PCR 的倍增寄存器为 3，符合理论计算（驱动根据周期时间进行倍增值计算，无需手动设置）。

③ 排查 PWM 的资源冲突状态

关注 Linux 端的资源使用，检测该 io 是否在 linux 端被使用（可检测 linux 端最终生成的.sunxi.dts 是否有 io 使用冲突路径在指令 cout（out/mr527/evb/openwrt）下），linux 是否也有使用到 rtos 端适配的 PWM 资源，避免两系统同时使用同一个 io 或 PWM，否则两系统相互影响。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。